

Project Report: Bloxorz Solver

Computation II: Algorithms & Data Structures

NOVA Information Management School (NOVA IMS)

May 2026

Group 14 Members:

Mohammad Ammaar

LEON APAYDIN

Amir Namatov

Oghenemega Ogugu

1. Introduction & Overview

This project focuses on the design and implementation of an automated solver for the puzzle game *Bloxorz*. The primary objective was to apply core computational concepts—such as graph traversal, state representation, and heuristic search—to navigate a $2 \times 1 \times 1$ block across a grid-based environment to a target goal.

2. System Architecture

We adopted a modular architecture inspired by the Model-View-Controller (MVC) pattern to ensure a clear separation of concerns:

- **Entry Point (Main):** Acts as the orchestrator, managing the application loop, main menus, level selection, and routing user choices to either manual play or AI solvers.
- **Model (Board, Block, State):** Manages the grid matrix, the physical movement mechanics, and the immutable mathematical representation of the block.
- **Controller (Game):** Acts as the core engine, handling step-by-step input, move history, and status checks (win/lose).
- **Solvers (Basic & A*):** The "intelligence" layer that computes optimal paths using BFS, DFS, and A*.
- **View (UI):** A console-based visualizer using `colorama` for real-time feedback.

3. State Representation & Mechanics

A critical design decision was the implementation of the `State` class as a `NamedTuple`. This ensures that every position of the block is **immutable** and **hashable**. In search algorithms, this allows for $O(1)$ lookup time in visited sets or priority queues, preventing redundant computation and infinite loops.

The block movement logic in `block.py` calculates new coordinates based on the current orientation (Upright, Horizontal, or Vertical). Every move is followed by a **normalization** step, where coordinates are sorted. This ensures that the same physical position always results in the same hashable state.

4. Search Algorithms & Complexity

The solver implements three distinct search strategies, allowing for a comparison between uninformed and informed search paradigms:

4.1 Breadth-First Search (BFS)

BFS explores the state space level-by-level. By using `collections.deque`, we ensure that adding and removing elements from the frontier are $O(1)$ operations. BFS guarantees the **optimal solution** (shortest path) for unweighted graphs.

Complexity: Time $O(b^d)$, Space $O(b^d)$.

4.2 Depth-First Search (DFS)

DFS explores branches to their maximum depth before backtracking. While space-efficient ($O(m)$), it does not guarantee the shortest path and may explore far deeper than necessary in Bloxorz levels.

4.3 Advanced Search (A*)

The A* algorithm optimizes search by combining the actual cost from the start (g) with an estimated cost to the goal (h). We utilized a heapq-based priority queue to always explore the most promising state first.

5. Heuristic Justification

For A* to remain optimal, the heuristic $h(s)$ must be **admissible** (never overestimate the true cost). We derived a modified Manhattan Distance for this purpose.

In Bloxorz, a block rolling from an upright position to a flat position covers 2 grid cells in a **single move**. Therefore, a standard Manhattan distance would overestimate the required steps. Our heuristic calculates the total Manhattan distance and divides it by 2.0 . This provides an optimistic but mathematically sound estimate:

$$h(s) = (|r_{\text{current}} - r_{\text{goal}}| + |c_{\text{current}} - c_{\text{goal}}|) / 2.0$$

6. Bonus Features

Beyond the core requirements, our team implemented several features to enhance the user experience and demonstrate advanced application of the concepts:

Feature	Technical Implementation
Colorama UI	ANSI escape sequences to differentiate the block, target, and terrain visually in the terminal.
Undo/ Replay	A LIFO stack (list) in the Game class that allows users to rollback states or watch a sequence of moves.
Par Score	A background BFS calculation that runs when a level starts to inform the player of the optimal number of moves.

7. Conclusion

The Bloxorz Solver successfully demonstrates the efficiency of **Informed Search (A*)** over uninformed methods. By strictly adhering to admissibility in our heuristic and leveraging hashable state structures, we achieved a high-performance solver capable of solving complex levels in milliseconds. The modular Python implementation ensures the code is maintainable and extensible for future level designs.